Lab Assignment 2: Recurrent Neural Network (RNN)

Name	Krushna Prashant Warule	**Dept**	Computer
Roll No	_____	**Div**	B
Subject	Advance Data Science	**Assignment No**	2
GitHub	https://github.com/krushnawarule/Advance_Data_Science_Lab_SCOE/tree/main/Assignment_2_RNN		

1. Aim

To implement an RNN model in Python to perform a sequence-based prediction task: the model takes a sequence of input values, learns the temporal relationships between them, and predicts the next value.

2. Theory

A Recurrent Neural Network processes a sequence one step at a time and keeps a **hidden state** that acts as memory: $h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$. The same weights are reused at every time step. Training uses **Backpropagation Through Time (BPTT)**, where the error is propagated backwards through all time steps; **gradient clipping** is used to avoid exploding gradients.

Task: given the previous 20 values of a series, predict the next value. **Model:** a vanilla RNN with 16 hidden units, implemented from scratch in NumPy (TensorFlow/Keras was not available), trained with Adam and MSE loss.

3. Dataset

Simulated time series of 1000 points: sin(0.1t) + 0.5 sin(0.037t) + Gaussian noise (std 0.1). Saved as `dataset.csv`. Sliding windows of length 20 give 980 samples: 784 for training and 196 for testing (time-ordered split).

4. Code (core part; full code in code.py)

```
p = {"Wx": np.random.randn(1, H) * 0.5, "Wh": np.random.randn(H, H) * 0.3,
     "b": np.zeros(H), "Wy": np.random.randn(H, 1) * 0.3, "by": np.zeros(1)}

def forward(X):
    hs = [np.zeros((len(X), H))]
    for t in range(X.shape[1]):                         # h_t = tanh(x_t*Wx + h_(t-1)*Wh + b)
        hs.append(np.tanh(X[:, t:t + 1] @ p["Wx"] + hs[-1] @ p["Wh"] + p["b"]))
    return hs[-1] @ p["Wy"] + p["by"], hs               # output from last hidden state

def backward(X, y, out, hs):                            # Backpropagation Through Time (BPTT)
    N = len(X); d = 2 * (out - y) / N
    g = {k: np.zeros_like(v) for k, v in p.items()}
    g["Wy"] = hs[-1].T @ d; g["by"] = d.sum(0); dh = d @ p["Wy"].T
    for t in reversed(range(X.shape[1])):
        dpre = dh * (1 - hs[t + 1] ** 2)
        g["Wx"] += X[:, t:t + 1].T @ dpre; g["Wh"] += hs[t].T @ dpre; g["b"] += dpre.sum(0)
        dh = dpre @ p["Wh"].T
    for k in g: g[k] = np.clip(g[k], -1, 1)             # gradient clipping
    return g
```

5. Output

Console output (output.txt):

```
Series length=1000 | window=20 | train samples=784 test samples=196
Trainable parameters: 305
Epoch   1/100  train MSE = 0.04852
Epoch  20/100  train MSE = 0.01751
Epoch  40/100  train MSE = 0.01812
Epoch  60/100  train MSE = 0.02621
Epoch  80/100  train MSE = 0.01473
Epoch 100/100  train MSE = 0.01629

Test RMSE (RNN)             : 0.1228
Test RMSE (naive last value): 0.1534
R2 score (RNN)              : 0.9767
```

Output images:

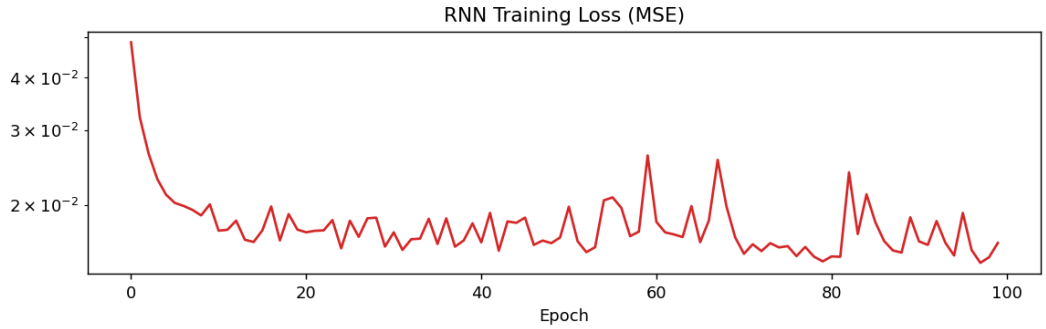


Fig 1: Training loss (MSE, log scale)

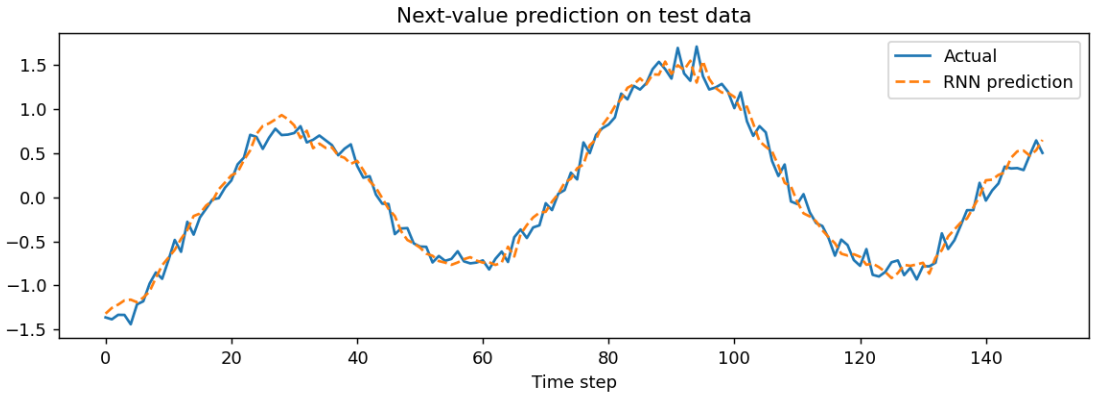


Fig 2: RNN predictions compared with actual values on test data

6. Conclusion

The RNN learned the temporal pattern of the series and reached a test RMSE of **0.1228** and an R^2 of **0.977**, better than the naive baseline that simply repeats the last value (RMSE 0.1534). Since the added noise has a standard deviation of 0.1, an RMSE close to 0.12 is near the best achievable result. This shows the hidden state successfully carries information about earlier time steps. For longer dependencies, LSTM or GRU cells would be preferable because a vanilla RNN suffers from vanishing gradients.

7. References

[1] J. Elman, 'Finding Structure in Time', Cognitive Science 14(2), 1990.

[2] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016 (Chapter 10: Sequence Modeling).

[3] R. Pascanu, T. Mikolov, Y. Bengio, 'On the difficulty of training recurrent neural networks', ICML 2013.

[4] D. Kingma, J. Ba, 'Adam: A Method for Stochastic Optimization', ICLR 2015.